

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Тема: «Сложные методы поиска»

По дисциплине «Основы алгоритмизации и программирования»

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

I. Поиск методом Бойера-Мура (Бойера-Мура-Хорспула)

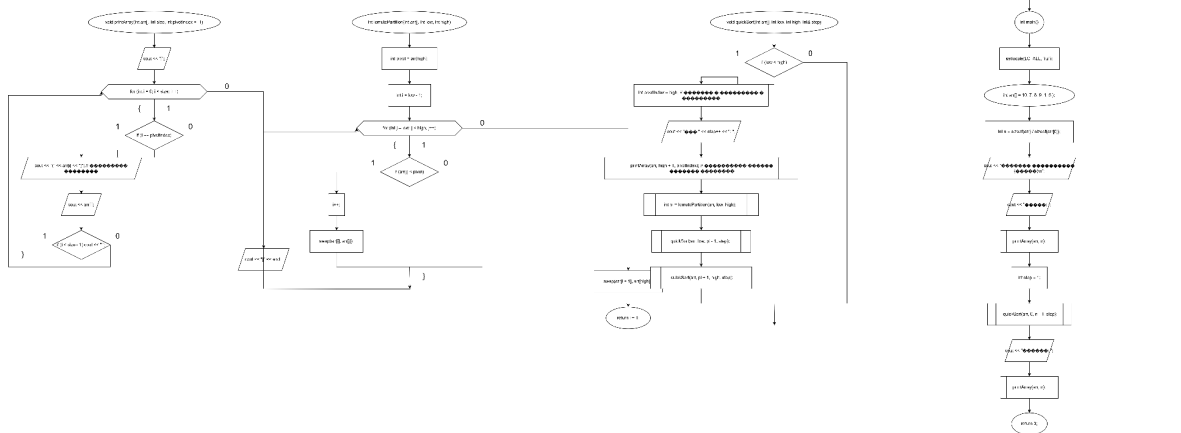
1. Постановка задачи

Найти подстроку в строке

2. Анализ задачи

- идем с конца шаблона
- для вычисления сдвига шаблона строим таблицу смещений
- таблица смещений ABCABD (в таблицу заносим наименьший сдвиг):
- сдвиг = длина шаблона - 1 - индекс символа в шаблоне
- $\text{shift} = \text{sublen} - 1 - \text{idx}$

3. Блок-схема



4. Код

```
#include <iostream>
#include <string>
```

```
using namespace std;
```

```
const int NO_OF_CHARS = 256;
```

```
void buildBadCharTable(string pattern, int badChar[]) {
    int m = pattern.length();
    for (int i = 0; i < NO_OF_CHARS; i++) {
        badChar[i] = m;
    }
    for (int i = 0; i < m - 1; i++) {
        int shift = m - 1 - i;
        badChar[(unsigned char)pattern[i]] = shift;
    }
}
```

```
void boyerMooreSearch(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();
    int badChar[NO_OF_CHARS];
    buildBadCharTable(pattern, badChar);
```

```
    cout << "\nТаблица сдвигов:" << endl;
    cout << "-----" << endl;
```

```

cout << "Символ | Сдвиг" << endl;
cout << "-----" << endl;
for (int i = 0; i < m; i++) {
    char c = pattern[i];
    if (badChar[(unsigned char)c] != m) {
        cout << " " << c << " | " << badChar[(unsigned char)c] << endl;
    }
}
cout << "-----" << endl;

int s = 0;
while (s <= (n - m)) {
    int j = m - 1;
    while (j >= 0 && pattern[j] == text[s + j]) {
        j--;
    }

    if (j < 0) {
        cout << "Найдено на позиции: " << s << endl;
        s += (s + m < n) ? 1 : m;
    }
    else {
        char badCharText = text[s + j];
        int shiftFromTable = badChar[(unsigned char)badCharText];
        int lastIndexInPattern = (m - 1) - shiftFromTable;
        int shift = j - lastIndexInPattern;
        if (shift < 1) shift = 1;

        cout << "Позиция " << s << ": несовпадение, сдвиг на " << shift << endl;
        s += shift;
    }
}
}

int main() {
    setlocale(LC_ALL, "ru");
    string text = "ABCAFDFAFCABD";
    string pattern = "ABCABD";

    cout << "Текст: " << text << endl;
    cout << "Шаблон: " << pattern << endl;

    boyerMooreSearch(text, pattern);

    return 0;
}

```

5. Результат выполнения программы

```

Текст:  ABCAFDFAFCABD
Шаблон: ABCABD

Таблица сдвигов:
-----
Символ | Сдвиг
-----
  А    |  2
  В    |  1
  С    |  3
  А    |  2
  В    |  1
-----

Позиция 0: несовпадение, сдвиг на 5
Позиция 5: несовпадение, сдвиг на 2
Найдено на позиции: 7

```

II. Алгоритм Кнута-Морриса-Пратта

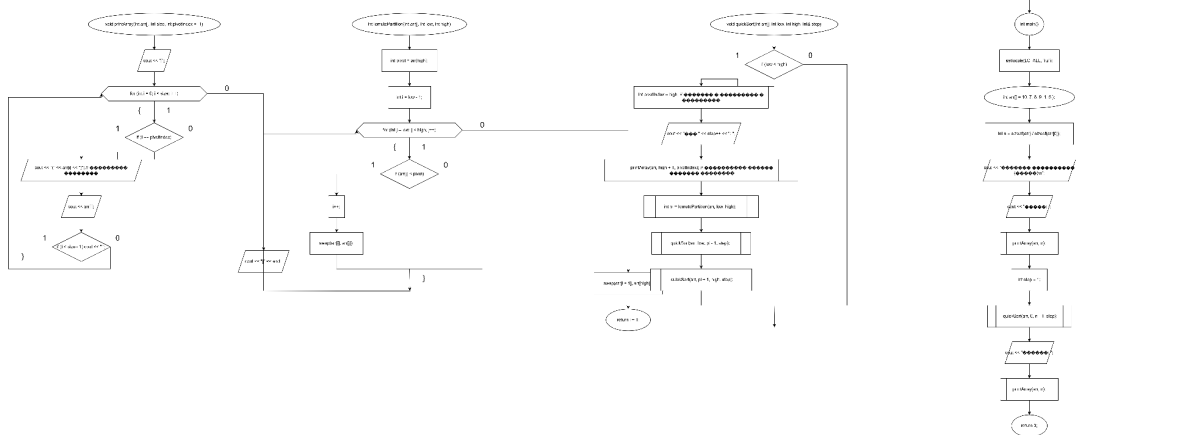
1. Постановка задачи

Найти подстроку в строке

2. Анализ задачи

- Префиксфункции: строится для каждого символа подстроки, именно на столько символов сдвигается шаблон подстроки на каждом шаге
- Значение префиксфункции - это максимальная длина совпадающих значений префикса и суффикса, которая заканчивается этим символом, для которого строится префикс функция.
- Длина массива, куда записываются значения префиксфункции равна длине подстроки.
- Для первого символа префиксфункция равна 0.
- рисунок
- d - массив значений префиксфункции
- слодные методы поиска, набор данных свой лр без номера, с отчетом

3. Блок-схема



4. Код

```
#include <iostream>
#include <string>
```

```
using namespace std;
```

```
void buildPrefixFunction(string pattern, int* d) {
    int m = pattern.length();
    d[0] = 0;

    for (int i = 1; i < m; i++) {
        int j = d[i - 1];
        while (j > 0 && pattern[i] != pattern[j]) {
            j = d[j - 1];
        }
        if (pattern[i] == pattern[j]) {
            j++;
        }
        d[i] = j;
    }
}
```

```

void showProgress(const string& text, const string& pattern, int pos, int i, int j) {
    cout << "\nТекст:  " << text << endl;
    cout << "Позиция: ";
    for (int k = 0; k < pos; k++) cout << " ";
    cout << pattern << endl;
    cout << "          ";
    for (int k = 0; k < pos + i; k++) cout << " ";
    if (i >= 0 && i < pattern.length()) cout << "^";
    cout << endl;
}

void kmpSearch(string text, string pattern) {
    int n = text.length();
    int m = pattern.length();
    int* d = new int[m];

    buildPrefixFunction(pattern, d);

    cout << "\nТаблица префикс-функции:" << endl;
    cout << "-----" << endl;
    cout << "i | символ | d[i]" << endl;
    cout << "-----" << endl;
    for (int i = 0; i < m; i++) {
        cout << i << " | " << pattern[i] << " | " << d[i] << endl;
    }
    cout << "-----" << endl;

    cout << "\nПроцесс поиска:" << endl;
    int j = 0;
    int step = 1;

    for (int i = 0; i < n; i++) {
        cout << "\nШаг " << step++ << endl;
        cout << "Сравниваем text[" << i << "]==" << text[i] << " с pattern[" << j << "]==" << pattern[j] << "" << endl;

        showProgress(text, pattern, i - j, j, i);

        while (j > 0 && text[i] != pattern[j]) {
            cout << "Несовпадение, j = d[" << j - 1 << "] = " << d[j - 1] << endl;
            j = d[j - 1];
        }

        if (text[i] == pattern[j]) {
            cout << "Совпадение!" << endl;
            j++;
        }
        else {
            cout << "Несовпадение, j остается 0" << endl;
        }

        if (j == m) {
            cout << "\nНайдено на позиции " << (i - m + 1) << endl;
            showProgress(text, pattern, i - m + 1, 0, m - 1);
            j = d[j - 1];
        }

        cout << "Текущее значение j: " << j << endl;
    }

    delete[] d;
}

```

```

int main() {
    setlocale(LC_ALL, "ru");
    string text = "ABCAFDFAFCABD";
    string pattern = "ABCABD";

    cout << "Текст: " << text << endl;
    cout << "Шаблон: " << pattern << endl;

    kmpSearch(text, pattern);

    return 0;
}

```

5. Результат выполнения программы

```

Текст:   ABCAFDFAFCABD
Шаблон:  ABCABD

```

Таблица префикс-функции:

i	символ	d[i]
0	A	0
1	B	0
2	C	0
3	A	1
4	B	2
5	D	0

Процесс поиска:

Шаг 1

Сравниваем text[0]='A' с pattern[0]='A'

Текст: ABCAFDFAFCABD

Позиция: ABCABD

^

Совпадение!

Текущее значение j: 1

Шаг 2

Сравниваем text[1]='B' с pattern[1]='B'

Шаг 12

Сравниваем text[11]='B' с pattern[4]='B'

Текст: ABCAFDFAFCABD

Позиция: ABCABD

^

Совпадение!

Текущее значение j: 5

Шаг 13

Сравниваем text[12]='D' с pattern[5]='D'

Текст: ABCAFDFAFCABD

Позиция: ABCABD

^

Совпадение!

Найдено на позиции 7

Текст: ABCAFDFAFCABD

Позиция: ABCABD

^

Текущее значение j: 0